

GNN Node Classification on Facebook Large Page-Page Network

1. Project Overview

This project implements a **Mixed Graph Neural Network (GNN)** for **semi-supervised multi-class node classification** using the **Facebook Large Page-Page Network Dataset**. The goal of the project is to predict the class of each node (page) based on 128-dimensional feature vectors and the graph structure connecting the nodes.

Additionally, the node embeddings learned by the GNN model are visualized using t-SNE to demonstrate how well the model clusters nodes of the same class.

2. Requirements

To install the newest libraries, run:

```
pip install torch torch-geometric numpy matplotlib scikit-learn
```

Dependencies

- Python: 3.8.10
- PyTorch: 2.4.1
- torch-geometric: 2.6.1
- NumPy: 1.26.4
- Matplotlib: 3.8.4
- scikit-learn: 1.5.1

Run with these specific versions I used to ensure reproducibility:

```
pip install torch==2.4.1 torch-geometric==2.6.1 numpy==1.26.4 matplotlib==3.8.4  
scikit-learn==1.5.1
```

3. Model Structure

The implemented model is a multi-layer **Mixed GNN** combining layers of **GCN**, **GAT**, and **GraphSAGE**:

- **Input Layer:** 128-dimensional input features per node.
- **Hidden Layer:** Four layers with 128 hidden units and 4 attention heads.
 - 2 GCNConv layers.
 - 2 GATConv layers with 4 attention heads.
 - 2 GraphSAGE layers.
- **Output Layer:** Provides logits for multi-class classification.

Hyperparameters (Best-Performing, accuracy:59.97%):

- Optimizer: AdamW
- Learning Rate: 0.001
- Weight Decay: 0.0001
- Hidden Dimensions: 128
- Number of Layers: 6 (2 GCN, 2 GAT, 2 GraphSAGE)
- Dropout: 0.3
- Early Stopping: 50 epochs patience
- learn rate scheduler: factor 0.6, patience 10

4. How Model Works

My model implements a a Mixed GNN that combines layers of **GCN**, **GAT**, and **GraphSAGE** for **semi-supervised node classification**. The model uses the graph structure and node features to classify nodes into predefined categories. Here's a brief overview of how the model works:

Node Features and Graph Structure:

- Each node in the graph has a **128-dimensional feature vector** representing its attributes (e.g., properties of a Facebook page).
- The graph is represented by its **edges**, which define the connections between nodes (e.g., links between Facebook pages).

Graph Attention and Convolution Mechanism:

- The model uses a **Graph Attention Network (GAT)**, which computes attention scores for each node's neighbors, and **Graph Convolution (GCN)** and **GraphSAGE** layers to update node representations based on their neighbors.
- This allows the model to focus on the most important neighbors when updating each node's representation.

Learning Process:

- The model is trained to **minimize the classification error** on the labeled nodes. The training process uses **AdamW** optimizer with a learning rate of **0.001** and a weight decay of **0.0001**.
- I also use a **learning rate scheduler** and **early stopping** to avoid overfitting and help the model converge faster.

Evaluation:

- The model is evaluated on a separate test set, and its performance is visualized using **t-SNE** to project the high-dimensional node embeddings onto a 2D space.

5. Run the Code

5.1. Train the Model

The training script (`train.py`) trains the GNN model and saves the trained model weights in `gnn_model.pth`. You can run the training script as follows:

```
python train.py
```

This script:

- Load the **Facebook Large Page-Page Network Dataset**
- Initialize a GNN model with **128 input features**, **GCN**, **GAT**, and **GraphSAGE** layers.
- Random Seed: 42 for reproducibility
- Training is done with early stopping and learning rate scheduling (ReduceLROnPlateau).
- Train the GNN model for a number of epochs
- Save the trained model to *gnn_model.pth* for later prediction

5.2. Prediction and Evaluation

To evaluate the model and visualize the learned embeddings, run the prediction script (predict.py):

```
python predict.py
```

This script:

- Loads the pre-trained model.
- Predicts node classes and computes the accuracy on the test set.
- Visualizes node embeddings using **t-SNE**.

6. Dataset

The dataset I used is **Facebook Large Page-Page Network dataset**, where the features are in the form of 128 dim vectors. It consist of:

- Nodes: Facebook pages.
- Edges: Links between pages.
- Features: 128-dimensional vectors representing the features of each page.
- Labels: The class of each node (page).

Preprocessing and Data Splitting

Preprocessing:

The original dataset was preprocessed by:

Loading the **.npz** file and extracting the relevant arrays.

Data Splitting:

The dataset is split into:

- **Training Set (80%)**: Used to train the model.
- **Test Set (20%)**: Used for final evaluation after training to assess model generalization.

The split is done using the `train_test_split` function from `scikit-learn`, with a fixed random seed (42) for reproducibility.

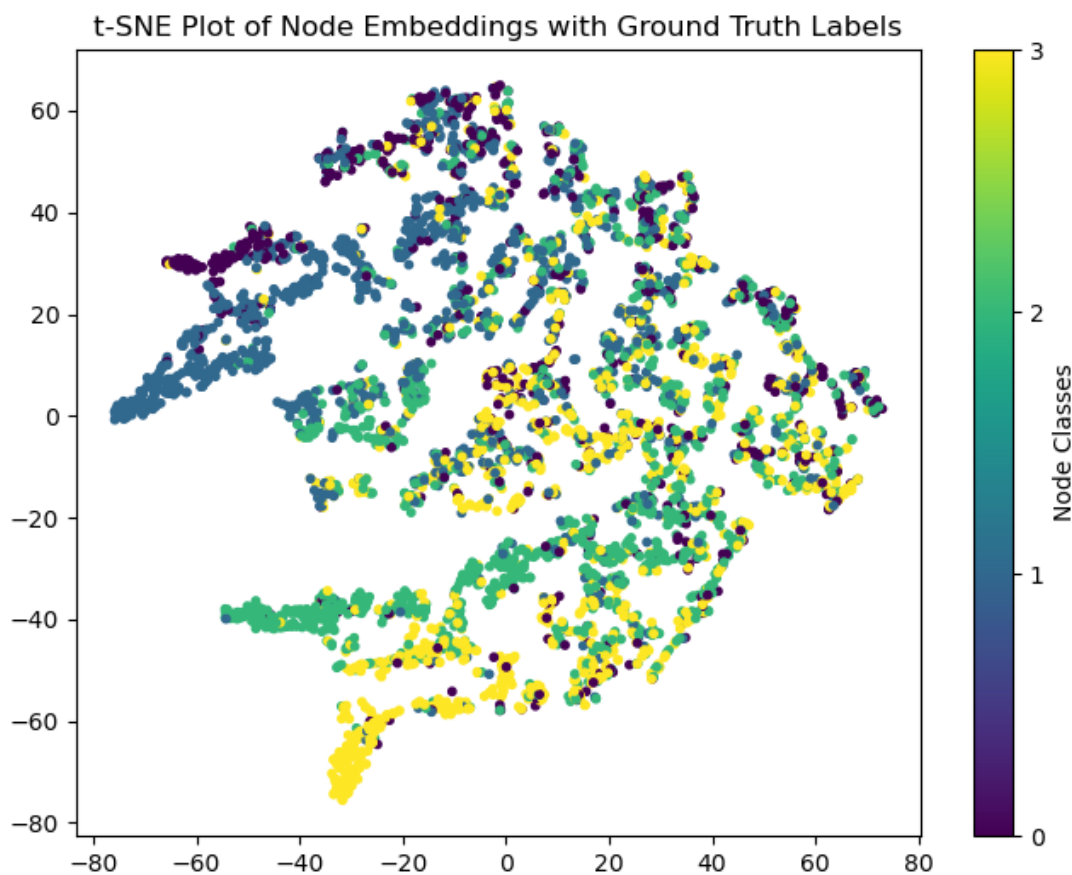
7. Result

7.1. Accuracy

The model achieved an accuracy of 59.97% on the test set after using the best hyperparameters (listed above).

7.2. Embedding Visualization (t-SNE)

Below is the **t-SNE visualization** of the learned node embeddings. Each color represents a different class, and the clusters show how the GNN model clusters nodes based on their learned embeddings.



In this **t-SNE Visualization**:

- While some distinct clusters are visible, there is still overlap between different classes, indicating that the model's representations are not fully separated for all classes.
- The overlapping regions suggest that the model has difficulty differentiating certain classes based on the learned embeddings, potentially due to:
 - Limited feature information in the dataset.
 - The inherent difficulty and possible noise in the task.
 - The model's architecture, which could be further optimized.

8. Drawbacks and Future Improvement

8.1. Model Performance and Scalability

- **Accuracy Ceiling:** 59.95% was the highest accuracy achieved despite extensive hyperparameter tuning.
- **Training Time:** Adding more layers increased training time without significantly improving accuracy.

8.2. Future Improvement

- **Ensemble Learning:** Try ensembling multiple models for higher accuracy..
- **Additional Layers:** Explore adding more attention heads or hidden layers with suitable hyperparameter settings.
- **Hyperparameter Search:** Search (random, grid...) for more optimal learning rates or dropout rates could improve performance.

9. Reference

1. Kipf, T. N., & Welling, M. (2016). Semi-Supervised Classification with Graph Convolutional Networks. arXiv preprint arXiv:1609.02907.
2. Facebook Large Page-Page Network Dataset: [Data Source](#), and the [Paritally proecessed one](#)

10. License

This project is licensed under the Apache License - see the [License](#) file for details.